

```
sted Loop :
-----
If we write a loop inside another loop then it is called Nested loop.
Example :
    for(int i=1; i<=4; i++)
    {
        //Nested loop
        for(int j=1; j<=7; j++)
        {
            IO.println("Day "+j);
        }
    }

* While working with nested loop, with each value of outer loop, the entire inner loop will be executed.

* In the above example, with each value of i, the value of j will be executed 7 times.

NestedLoop.java
-----
void main()
{
    int week = 4;
    int days = 7;

    for(int i=1; i<=week; i++) //i=1 1<=4 i++
    {
        IO.println("Week : "+i);    //Week 1

        for(int j=1; j<=days; j++)
        {
            IO.println("Day : "+j);
        }
        IO.println();
    }
}

package com.ravi.one_diemensional_basic;
import java.util.Arrays;

//WAP to sort the array elements in ascending order [Bubble sort]

public class ArrayDemo10 {

    public static void main(String[] args)
    {
        int arr[] = {70,10,80,90,20,30,50}; //length = 7
        IO.println("Original Data :"+Arrays.toString(arr));

        // i   j                i>j -> swap
        //70,10,80,90,20,30,50

        int temp = 0;
        for(int i=0; i<arr.length; i++)
        {
            for(int j=i+1; j<arr.length; j++)
            {
                //Swapping logic
                if(arr[i] > arr[j])
                {
                    temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }

        IO.println("Array elements in ascending order :"+Arrays.toString(arr));

    }
}

package com.ravi.one_diemensional_basic;

//Q10) WAP to search an element in the existing array based on the
//index position.

public class ArrayDemo11
{
    public static void main(String[] args)
    {
        int []arr = {2,3,9,2,8,7,3,2};

        int search = Integer.parseInt(IO.readLine("Enter the element to search : "));

        boolean isFound = false;

        for(int i=0; i<arr.length; i++)
        {
            if(arr[i]==search)
            {
                IO.println(search+" element found at "+i+" index ");
                isFound = true;
                break;
            }
        }

        if(!isFound)
        {
            System.err.println(search+" element is not available in the given array");
        }

    }
}

package com.ravi.one_diemensional_basic;
import java.util.Arrays;

//Q11) Find the duplicate elements in the Array and print the duplicate.

public class ArrayDemo12
{
    public static void main(String[] args)
    {
        int []arr = {10, 20, 30,10, 20, 40, 10, 50};
        IO.println("Original Array is :"+Arrays.toString(arr));

        for(int i=0; i<arr.length; i++)
        {
            for(int j=i+1; j<arr.length; j++)
            {
                if(arr[i]==arr[j])
                {
                    IO.println("Duplicate element found :"+arr[j]);
                    break;
                }
            }
        }

    }
}

package com.ravi.one_diemensional_basic;
import java.util.Arrays;

//Q12)* Insert a new element in the array at last position

class InsertAtLast
{
    public static int[] insertAtLastPosition(int []arr, int element)
    {
        int []newArray = new int[arr.length+1]; //10 20 30 40 0

        for(int i=0; i<arr.length; i++) //i=4
        {
            newArray[i] = arr[i];
        }

        newArray[newArray.length-1] = element;

        return newArray;
    }
}

public class ArrayDemo13
{
    public static void main(String[] args)
    {
        int []arr = {10,20,30,40};
        IO.println("Original Array :"+Arrays.toString(arr));

        int element = 50;
        int[] newArray = InsertAtLast.insertAtLastPosition(arr, element);
        IO.println("Array element after insertion :"+Arrays.toString(newArray));

    }
}

package com.ravi.one_diemensional_basic;
import java.util.Arrays;

//Q13) Insert a new element in the array at middle position.

class MiddleInsertion                //10 20 30 40 50 60
{
    public static int[] insertAtTheMiddle(int []arr, int element)
    {
        int []newArray = new int[arr.length + 1]; // 10 20 30 99 0 0 0

        //Find the middle position of the array
        int middle = arr.length / 2;

        //Copy the array elements upto middle position
        for(int i=0; i<middle; i++)
        {
            newArray[i] = arr[i];
        }

        newArray[middle] = element;

        //Copy remaining elements data
        for(int i=middle; i<arr.length; i++)
        {
            newArray[i+1] = arr[i];
        }

        return newArray;
    }
}

public class ArrayDemo14 {

    public static void main(String[] args)
    {
        int arr[] = {10,20,30, 40,50,60,70};    //7 /2

        IO.println("Original Array :"+Arrays.toString(arr));

        int element = 99;

        int[] newArray = MiddleInsertion.insertAtTheMiddle(arr, element);

        IO.println("Array elements after insertion :"+Arrays.toString(newArray));

    }
}

package com.ravi.one_diemensional_basic;
import java.util.Arrays;

//Q14) **Program to merge two array elements into a single array.

class MergeArray
{
    public static int[] mergeArrayElements(int []x, int []y)
    {
        int []newArray = new int[x.length + y.length]; //10 20 30 40

        int position = 0;

        for(int element : x)
        {
            newArray[position] = element;
            position++;
        }

        for(int element : y)
        {
            newArray[position] = element;
            position++;
        }

        return newArray;
    }
}

public class ArrayDemo15
{
    public static void main(String[] args)
    {
        int []val1 = {10,20,30,40,50};
        int []val2 = {100,200,300,400,500};

        int []mergedArray = MergeArray.mergeArrayElements(val1, val2);
        IO.println("After merging elements are :"+Arrays.toString(mergedArray));

    }
}

package com.ravi.one_diemensional_basic;

//Finding the unique elements in the array.

public class ArrayDemo16
{
    void main()
    {
        int []arr = {10,20,30,40,20,10};

        for(int i=0; i<arr.length; i++)
        {
            boolean isUnique = true;

            for(int j=0; j<arr.length; j++)
            {
                if(arr[i]==arr[j] && i!=j)
                {
                    isUnique = false;
                    break;
                }
            }

            if(isUnique)
            {
                IO.println("Unique element is :"+arr[i]);
            }
        }
    }
}
```